

1 Model

We consider a team task-allocation problem over a finite planning horizon. A planner must assign a set of tasks to a set of agents. Let I denote the set of tasks and let J denote the set of agents. Each task $i \in I$ can be assigned to at most one agent $j \in J$. The planner's decision is therefore not only whether a task should be carried out, but also which agent should perform it.

When task i is assigned to agent j , it produces a return and consumes workload. We let R_{ij} denote the expected return generated when agent j performs task i . This return may reflect the value created by completing the task, such as productivity gains, service value, or project contribution. We also let S_{ij} denote the random workload required when agent j performs task i . This workload can represent uncertain time, effort, attention, or cognitive demand. The same task may therefore lead to different expected returns and different workload realizations depending on the assigned agent.

Each agent $j \in J$ has a limited workload capacity, denoted by C_j . This capacity represents the maximum amount of work that the agent can reasonably handle during the planning horizon. Since actual workload is uncertain, a task assignment that looks acceptable in expectation may still overload an agent in some realizations. For this reason, the planner does not require workload feasibility in every possible realization. Instead, the planner allows a small probability of overload. Let $\alpha \in (0, 1)$ denote the maximum allowable overload probability for each agent. A smaller value of α means that the planner is more conservative and places more emphasis on protecting agents from overload risk.

In addition to controlling overload risk, the planner also wants to avoid highly uneven workload allocations across agents. A task assignment that places too much work on one agent and too little on another may increase fatigue, reduce sustainability, and create operational imbalance, even if the total expected return is high. To capture this idea, the model includes a workload-balance term in the objective function. Let $\lambda \geq 0$ denote the weight placed on workload imbalance. When λ is larger, the planner places more importance on balancing workload across agents. When $\lambda = 0$, the planner only maximizes return subject to the chance constraints.

To represent workload uncertainty, we approximate the random workload by a finite set of scenarios. Let Ω denote the set of workload scenarios, and let $\omega \in \Omega$ index a scenario. For each scenario ω , let p_ω denote its probability, where

$$\sum_{\omega \in \Omega} p_\omega = 1. \quad (1)$$

For each task-agent pair (i, j) and scenario ω , let S_{ij}^ω denote the realized workload of task i for agent j under scenario ω .

The planner's main assignment decision is represented by the binary variable

$$x_{ij} = \begin{cases} 1, & \text{if task } i \text{ is assigned to agent } j, \\ 0, & \text{otherwise,} \end{cases} \quad \forall i \in I, \forall j \in J. \quad (2)$$

Because each task can be assigned to at most one agent, the model will require

$$\sum_{j \in J} x_{ij} \leq 1, \quad \forall i \in I. \quad (3)$$

To implement the chance constraint in a scenario-based form, we introduce another binary variable:

$$y_{j\omega} = \begin{cases} 1, & \text{if agent } j \text{ is allowed to violate capacity in scenario } \omega, \\ 0, & \text{otherwise,} \end{cases} \quad \forall j \in J, \forall \omega \in \Omega. \quad (4)$$

This variable allows the model to mark a scenario as a violating scenario for agent j . However, the total probability of such violating scenarios must remain small. Specifically, the model will require the total probability of scenarios with $y_{j\omega} = 1$ to be no larger than α for each agent.

To measure how heavily each agent is loaded relative to that agent's own capacity, we define the expected workload utilization of agent j by

$$U_j = \frac{1}{C_j} \sum_{i \in I} \left(\sum_{\omega \in \Omega} p_\omega S_{ij}^\omega \right) x_{ij}, \quad \forall j \in J. \quad (5)$$

Here, U_j is the expected workload assigned to agent j , normalized by the agent's capacity C_j . This normalization is useful because agents may have different capacities, so absolute workload levels alone may not provide a fair comparison.

To measure imbalance across agents, we introduce a nonnegative variable Z to represent the maximum pairwise difference in expected workload utilization. More precisely, Z is defined through the constraints

$$U_j - U_k \leq Z, \quad \forall j \in J, \forall k \in J. \quad (6)$$

Since these inequalities hold for all ordered pairs (j, k) , the variable Z becomes an upper bound on the largest utilization gap across agents. A smaller value of Z therefore means a more balanced workload allocation.

Using the notation defined above, the planner solves the following optimization problem:

$$\max_{x,y,U,Z} \sum_{j \in J} \sum_{i \in I} R_{ij} x_{ij} - \lambda Z \quad (7)$$

$$\text{s.t.} \quad \sum_{j \in J} x_{ij} \leq 1, \quad \forall i \in I \quad (8)$$

$$\sum_{i \in I} S_{ij}^{\omega} x_{ij} \leq C_j + M y_{j\omega}, \quad \forall j \in J, \forall \omega \in \Omega \quad (9)$$

$$\sum_{\omega \in \Omega} p_{\omega} y_{j\omega} \leq \alpha, \quad \forall j \in J \quad (10)$$

$$U_j = \frac{1}{C_j} \sum_{i \in I} \left(\sum_{\omega \in \Omega} p_{\omega} S_{ij}^{\omega} \right) x_{ij}, \quad \forall j \in J \quad (11)$$

$$U_j - U_k \leq Z, \quad \forall j \in J, \forall k \in J \quad (12)$$

$$x_{ij} \in \{0, 1\}, \quad \forall i \in I, \forall j \in J \quad (13)$$

$$y_{j\omega} \in \{0, 1\}, \quad \forall j \in J, \forall \omega \in \Omega \quad (14)$$

$$U_j \geq 0, \quad \forall j \in J \quad (15)$$

$$Z \geq 0. \quad (16)$$

In this formulation, the objective function in (7) maximizes total expected return while penalizing workload imbalance. The first term measures the total expected value created by the task assignment. The second term penalizes large differences in expected workload utilization across agents. Constraint (8) ensures that each task is assigned to at most one agent. Constraint (9) imposes workload feasibility in each scenario unless that scenario is explicitly marked as a violating scenario through $y_{j\omega} = 1$. Constraint (10) limits the total probability of violating scenarios for each agent, which is the scenario-based implementation of the chance constraint. Constraint (11) defines the expected workload utilization of each agent, and constraint (12) links these utilization levels to the imbalance variable Z . Finally, constraints (13)–(16) define the domains of the decision variables.

The planner assigns tasks to agents. Those assignments determine both expected return and uncertain workload outcomes. The uncertain workload outcomes then determine whether each agent remains within capacity with sufficiently high probability. At the same time, the assignment also determines how evenly work is distributed across agents. The optimal solution therefore balances three forces: return creation, overload-risk control, and workload fairness.

2 Solution Methodology

The compact formulation in Section 1 can be solved directly by a mixed-integer solver for small and medium-sized instances. However, when the numbers of tasks, agents, and scenarios become large, the scenario-based chance constraints become the main computational bottleneck. In particular, the

formulation introduces one binary variable for each agent-scenario pair and one workload constraint for each agent-scenario pair. To exploit the structure of the problem more effectively, we solve the model by a Benders decomposition.

The workload-balance component depends only on expected workload and therefore can remain in the master problem. By contrast, the scenario-based chance-feasibility condition is naturally evaluated only after a candidate assignment has been fixed. This structure allows us to separate the assignment and balancing decisions from the scenario-based feasibility check.

For convenience, we first define the expected workload coefficient

$$\bar{S}_{ij} = \sum_{\omega \in \Omega} p_{\omega} S_{ij}^{\omega}, \quad \forall i \in I, \forall j \in J. \quad (17)$$

Using (17), the expected workload utilization of agent j can be written as

$$U_j = \frac{1}{C_j} \sum_{i \in I} \bar{S}_{ij} x_{ij}, \quad \forall j \in J. \quad (18)$$

2.1 Master problem

At iteration t , let $\mathcal{B}^t$ denote the set of Benders feasibility cuts that have been generated so far. The master problem is

$$\max_{x, U, Z} \sum_{j \in J} \sum_{i \in I} R_{ij} x_{ij} - \lambda Z \quad (19)$$

subject to

$$\sum_{j \in J} x_{ij} \leq 1, \quad \forall i \in I, \quad (20)$$

$$U_j = \frac{1}{C_j} \sum_{i \in I} \bar{S}_{ij} x_{ij}, \quad \forall j \in J, \quad (21)$$

$$U_j - U_k \leq Z, \quad \forall j \in J, \forall k \in J, \quad (22)$$

$$x_{ij} \in \{0, 1\}, \quad \forall i \in I, \forall j \in J, \quad (23)$$

$$U_j \geq 0, \quad \forall j \in J, \quad (24)$$

$$Z \geq 0, \quad (25)$$

$$x \in \mathcal{B}^t. \quad (26)$$

The master problem keeps the true objective function of the original model. It chooses a candidate assignment pattern and simultaneously evaluates the workload-balance term through the utilization variables U_j and the imbalance variable Z . The chance constraints are not imposed explicitly in the master problem. Instead, they are checked in a separate feasibility evaluation step after a candidate

assignment has been obtained.

2.2 Feasibility evaluation subproblem

Suppose that the master problem returns a candidate assignment $\bar{x} = \{\bar{x}_{ij}\}$. For each agent $j \in J$ and scenario $\omega \in \Omega$, the realized workload under $\bar{x}$ is

$$L_j^\omega(\bar{x}) = \sum_{i \in I} S_{ij}^\omega \bar{x}_{ij}, \quad \forall j \in J, \forall \omega \in \Omega. \quad (27)$$

Given $\bar{x}$, the smallest feasible value of the violation indicator in scenario ω for agent j is

$$\delta_{j\omega}(\bar{x}) = \begin{cases} 1, & \text{if } L_j^\omega(\bar{x}) > C_j, \\ 0, & \text{if } L_j^\omega(\bar{x}) \leq C_j, \end{cases} \quad \forall j \in J, \forall \omega \in \Omega. \quad (28)$$

Therefore, the total violation probability for agent j under assignment $\bar{x}$ is

$$P_j^{\text{viol}}(\bar{x}) = \sum_{\omega \in \Omega} p_\omega \delta_{j\omega}(\bar{x}), \quad \forall j \in J. \quad (29)$$

The candidate assignment $\bar{x}$ is chance-feasible if and only if

$$P_j^{\text{viol}}(\bar{x}) \leq \alpha, \quad \forall j \in J. \quad (30)$$

This feasibility check does not require solving another mixed-integer optimization model. Once $\bar{x}$ is fixed, the realized workload in each scenario is known, and the violation probability of each agent can be computed directly from (27)–(30). For this reason, the subproblem in our decomposition is a logical feasibility checker rather than a separate optimization problem.

2.3 Feasibility cut generation

If (30) holds for every agent, then $\bar{x}$ is feasible for the original model, and the algorithm terminates with an optimal solution because the master problem already contains the exact objective function.

If (30) is violated for some agent j , then the current assignment gives agent j too many tasks relative to the allowed overload probability. Let

$$I_j(\bar{x}) = \{i \in I : \bar{x}_{ij} = 1\} \quad (31)$$

denote the set of tasks assigned to agent j in the current candidate solution. A valid no-good

feasibility cut is

$$\sum_{i \in I_j(\bar{x})} x_{ij} \leq |I_j(\bar{x})| - 1. \quad (32)$$

Constraint (32) prevents the master problem from assigning exactly the same bundle of tasks to the same infeasible agent again.

In practice, a stronger cut can be obtained by identifying a smaller offending subset $B_j(\bar{x}) \subseteq I_j(\bar{x})$ such that

$$\sum_{\omega \in \Omega: \sum_{i \in B_j(\bar{x})} S_{ij}^\omega > C_j} p_\omega > \alpha. \quad (33)$$

If such a subset is available, then we add the stronger feasibility cut

$$\sum_{i \in B_j(\bar{x})} x_{ij} \leq |B_j(\bar{x})| - 1. \quad (34)$$

Constraint (34) excludes the specific task combination that causes excessive overload risk for agent j , while leaving more flexibility for the master problem than (32).

2.4 Algorithmic procedure

The overall solution procedure is iterative. We start with the master problem without any Benders feasibility cuts. At each iteration, we solve the master problem and obtain a candidate assignment $\bar{x}$. We then evaluate the chance-feasibility of $\bar{x}$ by computing the realized workload in every scenario and the resulting violation probability for each agent. If the candidate assignment is feasible, then it is optimal for the original problem. If the candidate assignment is infeasible, then we generate one or more feasibility cuts of the form (32) or (34), add them to the master problem, and solve the updated master problem again. The procedure continues until a chance-feasible assignment is found.

This decomposition is attractive for two reasons. First, it keeps the true objective function and the workload-balance structure in the master problem, so every master solution is evaluated with respect to the exact return-balance trade-off. Second, it moves the scenario-based chance-feasibility check into a separate logical step, which avoids carrying all agent-scenario binary variables explicitly inside the main optimization model. As a result, the method remains closely aligned with the managerial logic of the problem: the planner first proposes an assignment, and then checks whether that assignment is sufficiently safe under workload uncertainty.